

Chapitre IV : Les protocoles de sécurité : SSL/TLS

Le protocole SSL (Socket Secure Layer) fonctionne suivant un mode client/serveur. Il permet d'assurer les services de sécurité suivante aux clients TCP: l'authentification du serveur; les services de confidentialité; l'intégrité des services de connexion (Lamotte et al., 2005; Oppliger, 2009).

Le protocole SSL a été développé par Netscape Communication en 1994. Il a directement publié la version SSLv2 en 1995. Un certain nombre de lacunes ont été découverte dans cette version. Pour cette raison, la version 3 a été publiée en novembre 1996 et a permet de pallier ces lacunes. En mai 1996, l'Internet Engineering Task Force (IETF) a mis en place un groupe de travail appelé Transport Layer Security (TLS) afin de poursuivre les travaux de Netscape Communication. En janvier 1999 l'IETF a publié TLS 1.0 (considéré comme étant SSL 3.1) (Lamotte et al., 2005; Oppliger, 2009).

Les concepteurs de SSL ont décidé de créer un protocole distinct seulement pour la sécurité. En effet, ils ont ajouté une couche à l'architecture de protocole de l'Internet (IP). La figure 4.1 (Stephen, 2000) montre les principaux protocoles pour les communications Web. En bas on trouve le protocole Internet (IP). Ce protocole est responsable de l'acheminement des messages à travers des réseaux à partir de leur source jusqu'à leur destination. Au milieu on trouve le protocole TCP (Transmission Control Protocol) qui permet d'assurer la fiabilité de la communication. Au-dessus il opère l'HyperText Transfer Protocol (http) qui comprend les détails de l'interaction entre les navigateurs Web et les serveurs Web. Le SSL assure la sécurité en agissant comme un protocole de sécurité séparé, le SSL est introduit entre l'application HTTP et TCP comme le montre dans la figure 4.2 (Stephen, 2000).

Chapitre IV : Les protocoles de sécurité : SSL/TLS

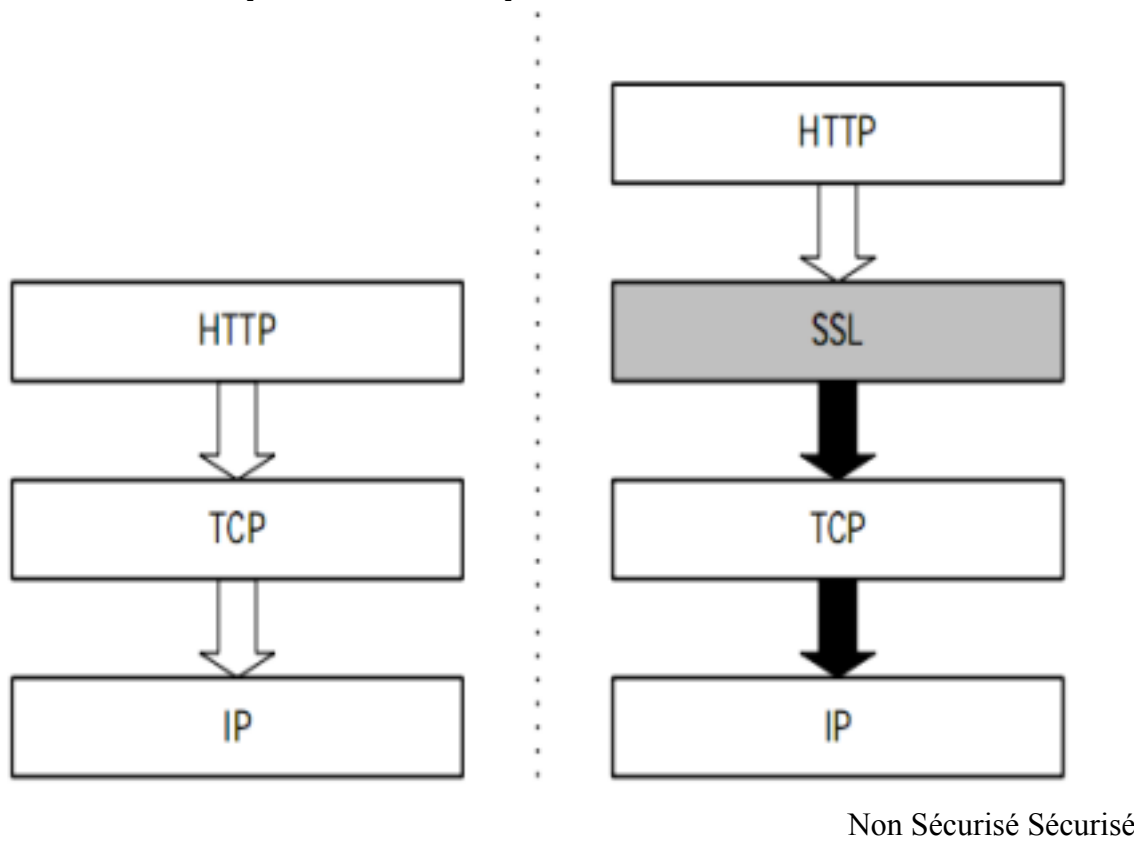


Figure 4.1 Les protocoles de communication Figure 4.2 SSL est une couche de protocole particulier assurant la sécurité (Stephen, 2000)

4.1. Architecture du protocole SSL

Le protocole SSL se compose de plusieurs protocoles comme l'illustre la figure 4.3: le protocole d'enregistrement (Record protocol), le protocole de poignée de main (Handshake Protocol), le protocole de changement des spécifications de chiffrement (Change Cipher Spec Protocol), le protocole d'alerte (Alert Protocol) et le protocole de données d'application (Stephen, 2000).

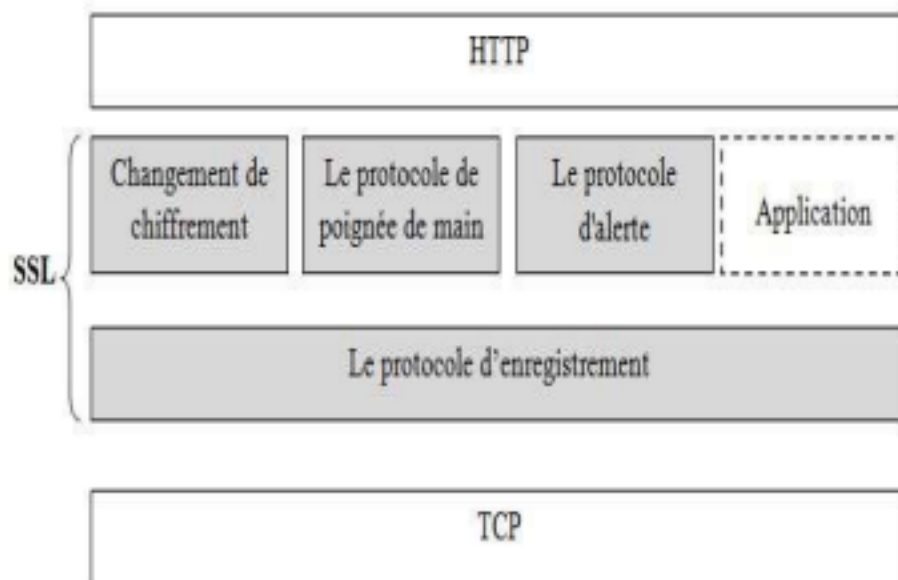


Figure 4.3 Les composants du protocole SSL (Stephen, 2000)

4.1.1. Le protocole d'enregistrement (Record protocol)

Le protocole d'enregistrement est utilisé pour l'encapsulation des données du protocole de la couche supérieure. Le traitement de protocole d'enregistrement est illustré à la figure 4.4. Il commence par la fragmentation des données de la couche supérieure en blocs de 214 octets ou moins (appelés des fragments). Ensuite, il compresse chaque fragment selon la méthode de compression spécifiée dans l'état de la session SSL. Puis il calcule un condensât (MAC) et il protège le fragment compressé et le condensât selon l'algorithme de chiffrement spécifié dans l'état de la session SSL. Enfin, le protocole d'enregistrement SSL ajoute un en-tête d'enregistrement SSL au fragment crypté pour obtenir l'enregistrement SSL. L'en-tête de l'enregistrement SSL comprend les trois champs suivants (figure 4.5) (Oppliger, 2009):

1. Le type du contenu qui occupe les premiers 8 bits : Il fait référence au protocole SSL de couche supérieure. Il y a quatre valeurs prédéfinies (Oppliger, 2009):
 - ❖ 20 : fait référence au protocole de changement des spécifications de chiffrement (Change Cipher Spec Protocol);
 - ❖ 21 : fait référence au protocole d'alerte ;
 - ❖ 22 : fait référence au protocole de poignée de main (Handshake Protocol);
 - ❖ 23 : se réfère au protocole des données d'application.

Chapitre IV : Les protocoles de sécurité : SSL/TLS

2. La version de protocole : se réfère à la version du protocole SSL utilisé. Il occupe les deux octets qui se composent de valeur major et mineure de numéro de version de SSL/TLS: 2.0 pour SSL, v2, 3.0 pour SSLv3 et 3.1 pour TLS.
3. Un champ d'une longueur de 16 bits qui se réfère à la longueur du fragment, chaque fragment est de taille 2¹⁴ octet ou inférieure après la décompression.

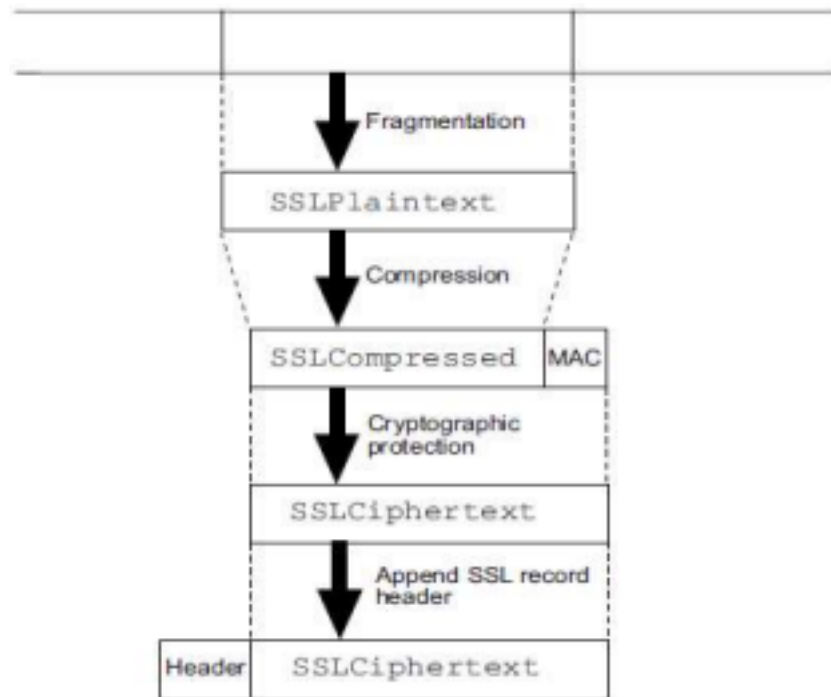


Figure 4.4

Le processus du protocole d'enregistrement SSL (Oppliger, 2009)

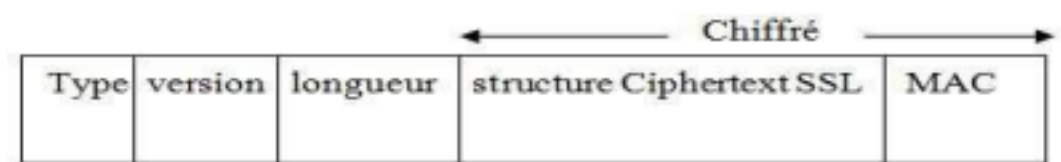


Figure 4.5 L'entête d'un enregistrement SSL (Oppliger, 2009)

4.1.2. Le protocole de poignée de main (Handshake Protocol)

Le protocole de négociation SSL permet à un client et un serveur de s'authentifier mutuellement, de négocier les paramètres confidentiels comme les algorithmes de chiffrement et les méthodes de compression et d'établir les clés qui seront utilisées dans les algorithmes choisis. La figure 4.6 illustre le format des messages de négociation à travers des enregistrements et indique que plusieurs messages de prise

Chapitre IV : Les protocoles de sécurité : SSL/TLS

de contact peuvent être (et ils sont fréquemment) combinés en un message de couche d'enregistrement unique. La partie fortement encadrée d'un message fait référence au message (s) de négociation SSL, alors que les 5 principaux octets se réfèrent à l'entête d'enregistrement SSL. Cet entête, à son tour, comporte toujours une valeur de type (22) de 1 octet (se référant au Protocole SSL Handshake), une valeur de la version 3,0 de 2 octets, et une valeur de longueur 2 octets se référant à la longueur en octets de la partie restante du disque SSL (Oppliger, 2009).

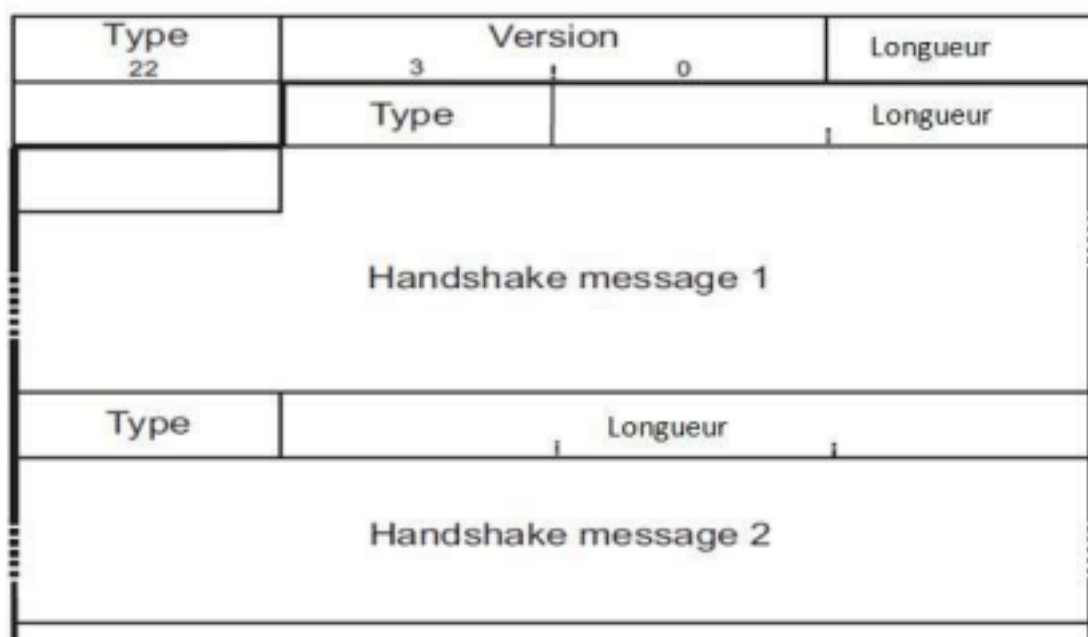


Figure 4.6 La structure d'un message de protocole de négociation SSL (Oppliger, 2009)

❖ Les étapes de la négociation:

Le protocole et les flux des messages sont illustrés à la figure 4.7. Les messages qui sont écrits entre crochets sont facultatifs ou dépendants de la situation, ce qui signifie qu'ils ne sont pas toujours envoyés. Le protocole de négociation SSL comprend quatre ensembles de messages qui sont échangés entre le client et le serveur. Chaque ensemble est typiquement transmis dans un segment TCP séparé (Oppliger, 2009). □
Le premier ensemble de messages est envoyée par le client au serveur. Il comporte uniquement un message de « CLIENTHELLO ». Ceci indique que le client veut établir une connexion sécurisée (Oppliger, 2009).

Chapitre IV : Les protocoles de sécurité : SSL/TLS

□ Le deuxième ensemble de messages comporte 2-5 messages qui sont envoyés par le serveur au client (Oppliger, 2009):

1. Un message de « SERVERHELLO » est envoyé comme réponse de message « CLIENTHELLO » pour lui indiquer qu'il a bien reçu son message.
2. Si le serveur s'est authentifié (ce qui est généralement le cas), il peut envoyer un message de « CERTIFICATE » au client.
3. Dans certaines conditions le client demande de transmettre des clés sécurisées, le serveur peut envoyer un message de «SERVERKEYEXCHANGE » au client.
4. Si le serveur exige que le client s'authentifie avec un certificat, il peut envoyer un message de « CERTIFICATE REQUEST » au client. 5. Enfin, le serveur envoie un message de SERVERHELLODONE au client.



Figure 4.7 Le protocole de poignée de main (Handshake Protocol) (Oppliger, 2009)

A la fin des échanges des messages ClientHello et ServerHello, le client et le serveur achèveront la négociation d'une version de protocole et auront un identifiant de session (ID), l'algorithme de chiffrement, et une méthode de compression. En outre, deux valeurs aléatoires (ClientHello.random et ServerHello.random) sont générées et disponibles pour l'utilisation.

❖ Le troisième ensemble de messages comprend 3-5 messages qui sont de nouveaux envoyés à partir du client vers le serveur:

1. Si le serveur envoie un message de « CERTIFICATE REQUEST », le

client envoie un message de « CERTIFICATE » au serveur.

Chapitre IV : Les protocoles de sécurité : SSL/TLS

2. Dans l'étape principale du protocole, le client envoie un message de « CLIENTKEYEXCHANGE » au serveur. Le contenu de ce message dépend de l'algorithme d'échange de clé en cours d'utilisation.
3. Si le client a envoyé un certificat au serveur, il doit également envoyer un message « CERTIFICATEVERIFY » au serveur. Ce message est signé numériquement avec la clé privée correspondant à la clé publique du certificat.
4. Le client envoie un message « CHANGECIPHERSPEC » au serveur (en utilisant Le protocole de changement des spécifications de chiffrement).
5. Le client envoie un message crypté « FINISHED » au serveur. ❖ le quatrième ensemble de messages comprend deux messages qui sont envoyés par le serveur vers le client :
 1. Le serveur envoie un autre message « CHANGECIPHERSPEC » au client.
 2. le serveur envoie un message (crypté) « FINISHED » au client. A ce moment, la négociation SSL est terminée; le client et le serveur peuvent commencer à échanger les données de la couche application (en utilisant le protocole SSL Application Data) (Oppliger, 2009).

4.1.3. Le protocole d'alerte

Le protocole d'alerte est utilisé pour notifier des avertissements ou des erreurs qui auraient pu se produire, par exemple si un certificat ne pouvait pas être vérifié. Le protocole SSL permet d'alerter les pairs de communication pour échanger des messages d'alerte. Chaque message d'alerte porte un niveau d'alerte et une description d'alerte (Oppliger, 2009):

- ❖ Le niveau d'alerte comprend 1 octet, où la valeur 1 signifie «avertissement» et la valeur 2 signifie « fatale ». Pour tous les messages d'erreurs pour lesquels le niveau d'alerte particulière n'est pas explicitement spécifié, l'expéditeur peut déterminer (d'après sa propre discrétion) si elle est fatale ou non. De même, si une alerte avec un niveau d'alerte d'avertissement est reçue, le récepteur peut décider à sa discrétion s'il doit le traiter comme une erreur fatale. ❖ La description de l'alerte comprend également 1 octet, où un code numérique se réfère à une situation

spécifique (Oppliger, 2009).

Page 7

Chapitre IV : Les protocoles de sécurité : SSL/TLS

4.1.4. Le protocole d'application des données

Le protocole d'application des données SSL permet aux pairs de communication d'échanger des données selon un protocole de couche d'application. Plus précisément, il prend les données d'application et il les transmet au protocole d'enregistrement SSL pour la fragmentation, la compression et la protection cryptographique (Oppliger, 2009).

